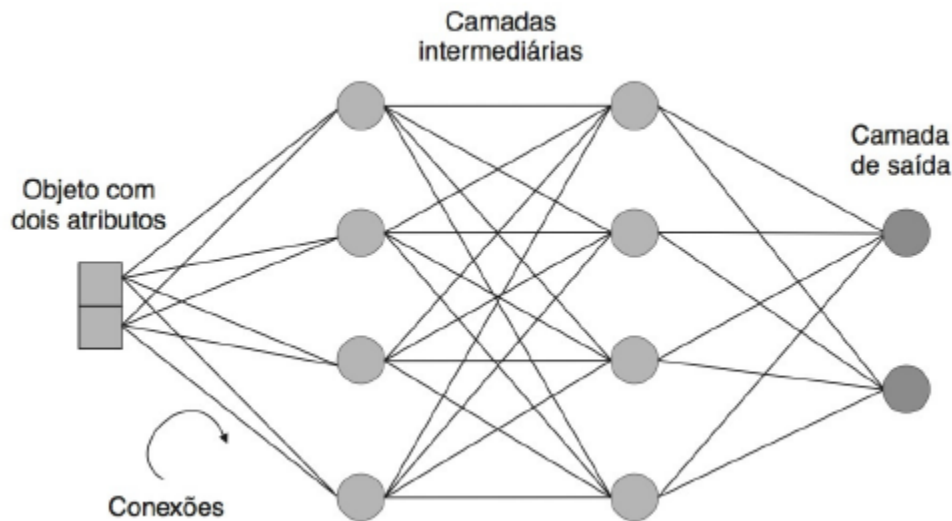


REDES NEURAIS BACKPROPAGATION

Cristiane Neri Nobre

MLP – *MultiLayer* Perceptron ou perceptron multicamadas

- Para resolver problemas não linearmente separáveis utilizando RNAs, a alternativa mais utilizada é adicionar uma ou mais camadas intermediárias.
- As redes do tipo MLP apresentam uma ou mais camadas intermediárias de neurônios e uma camada de saída.



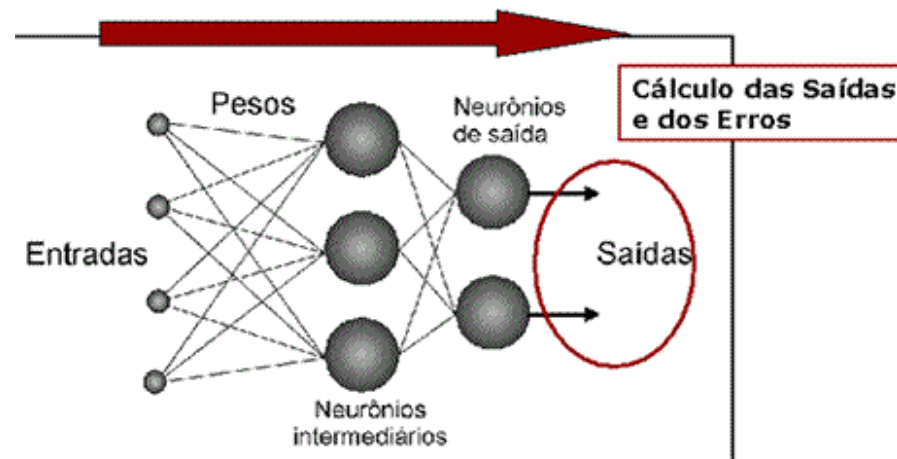
Algoritmo Backpropagation

- O **backpropagation** é um método de **otimização baseada em gradiente**. Ele ajusta os pesos da rede neural calculando como o erro varia quando cada peso muda um pouco
-
- A essência do algoritmo é o **gradiente descendente**, que depende das derivadas para saber **a direção e a intensidade** da mudança no erro.
- Sem derivadas, o algoritmo **não saberia se deve aumentar ou diminuir o peso**. Portanto, **a noção de derivada é essencial em todo backpropagation**.

Algoritmo Backpropagation

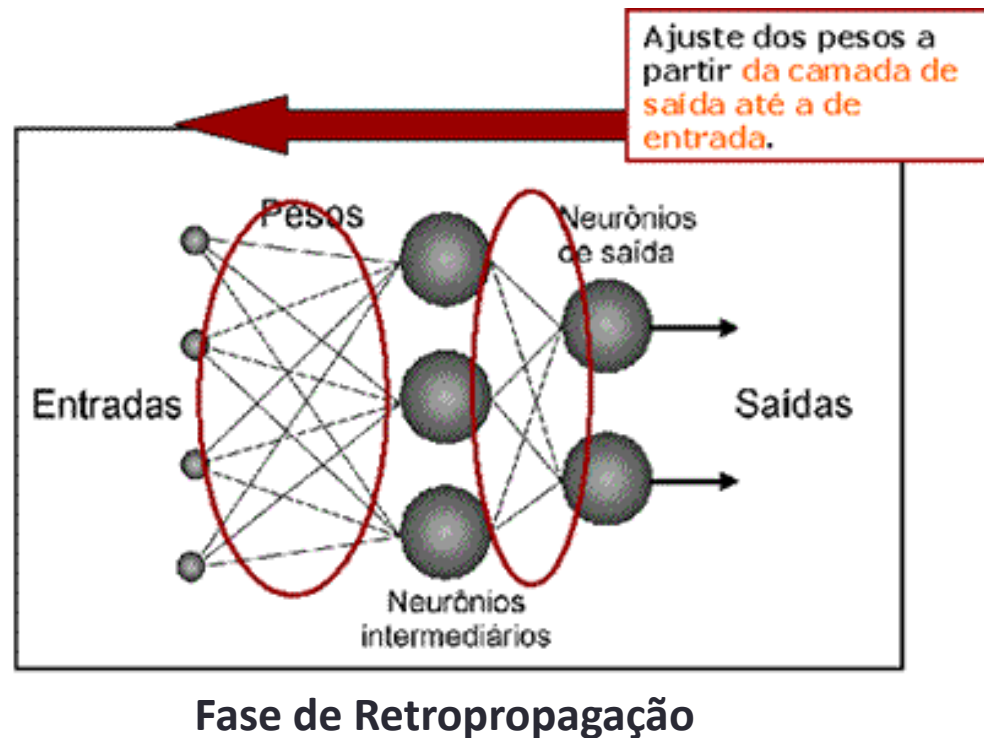
Baseia-se no Aprendizado Supervisionado por Correção de Erros, constituído de:

1º - Propagação (fase *forward*): Depois de apresentado o padrão de entrada, a resposta de uma unidade é propagada como entrada para as unidades na camada seguinte, até a camada de saída, onde é obtida a resposta da rede e o erro é calculado;



Algoritmo Backpropagation

2º - Retropropagação (fase *backward*) Desde a camada de saída até a camada de entrada, são feitas alterações nos pesos sinápticos.



Algoritmo Backpropagation

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

$$w_{jl}(t + 1) = w_{jl}(t) + \eta x^j \delta_l \quad (7.3)$$

Backpropagation - Inicialização

1 - Inicialização: Inicialize os pesos sinápticos e os bias aleatoriamente

Backpropagation – Computação para frente

2 - Computação para frente (propagação):

Depois de apresentado o exemplo do conjunto de treinamento $T = \{(x(n), d(n))\}$, sendo $x(n)$ a entrada apresentada à rede e $d(n)$ a saída desejada, calcule o valor da ativação v_j e a saída para cada unidade da rede, da seguinte forma:

$$v_j = \sum_{i=1}^m w_{ji} x_i + b x_0$$

para o cálculo do valor da ativação e

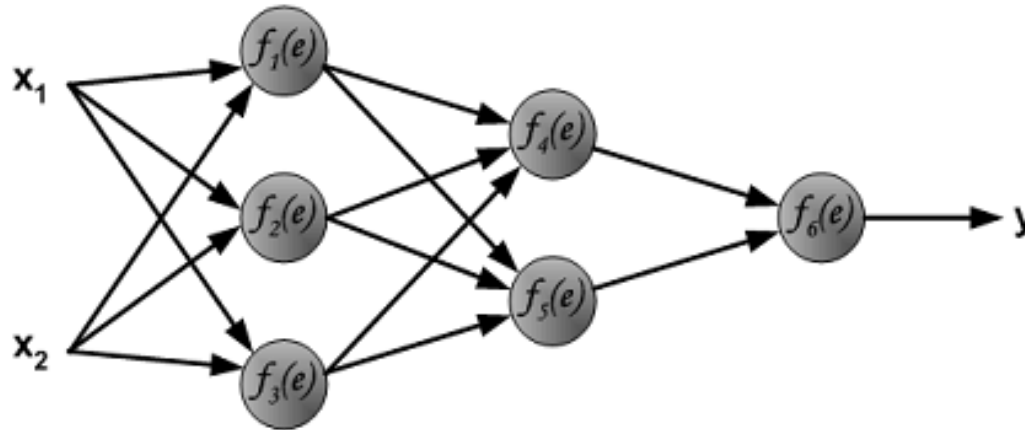
$$f(v) = \frac{1}{1 + e^{(-av)}}$$

para o cálculo da saída y da unidade k , utilizando a função sigmóide, como no exemplo, ou uma outra função, se necessário.

Utilize a saída das unidades de uma camada como entradas para a seguinte, até a última camada. A saída das unidades da última camada será a resposta da rede.

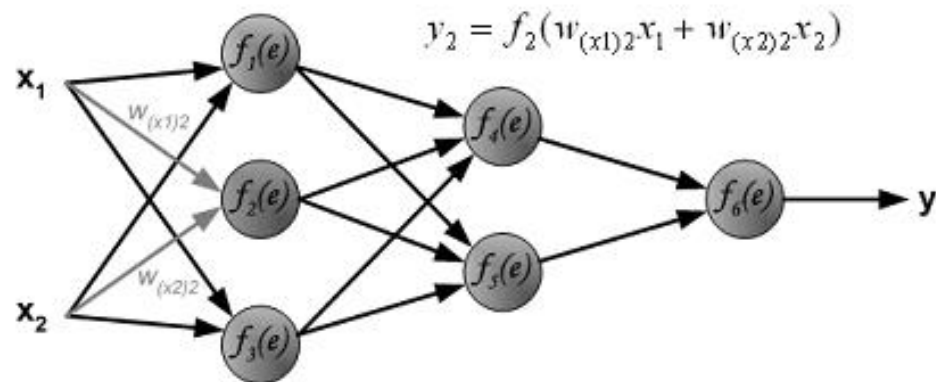
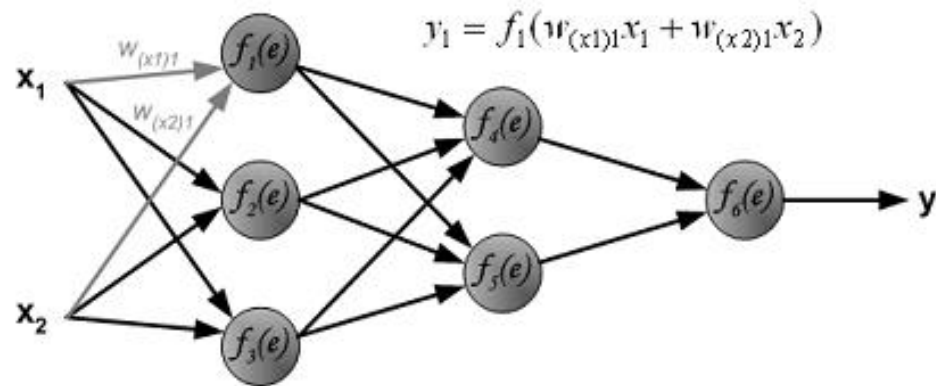
Backpropagation – Computação para frente

Exemplificação do passo 2:



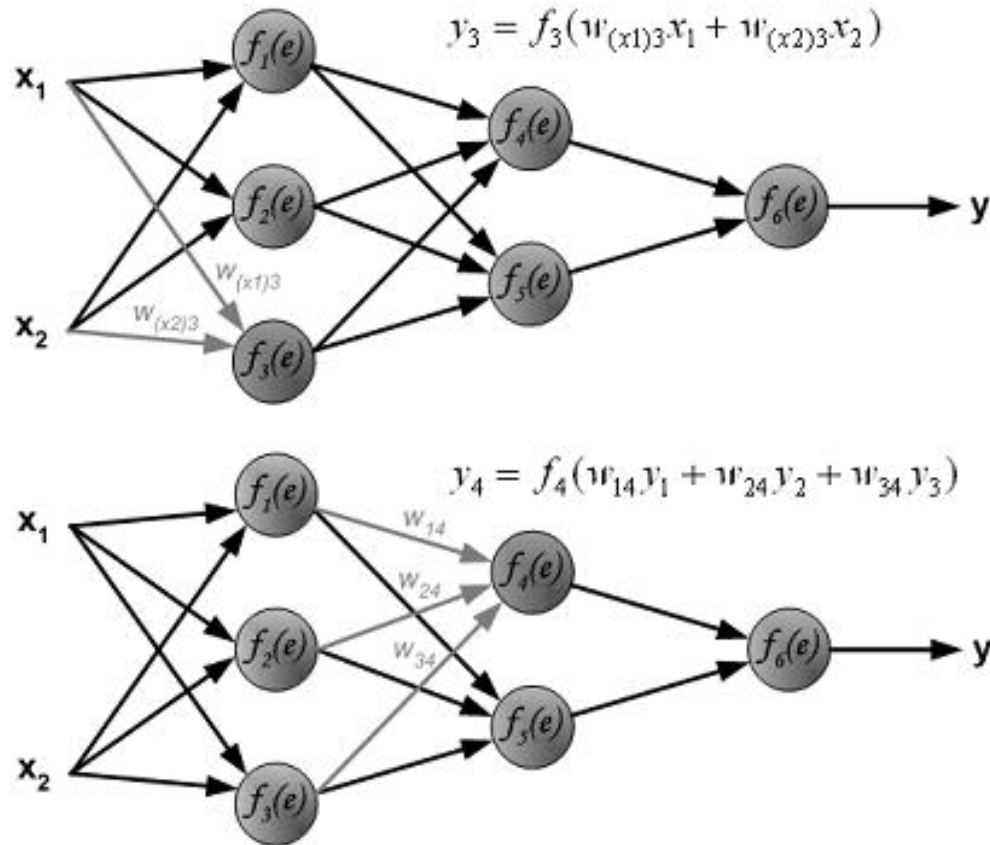
Backpropagation – Computação para frente

Exemplificação do passo 2:



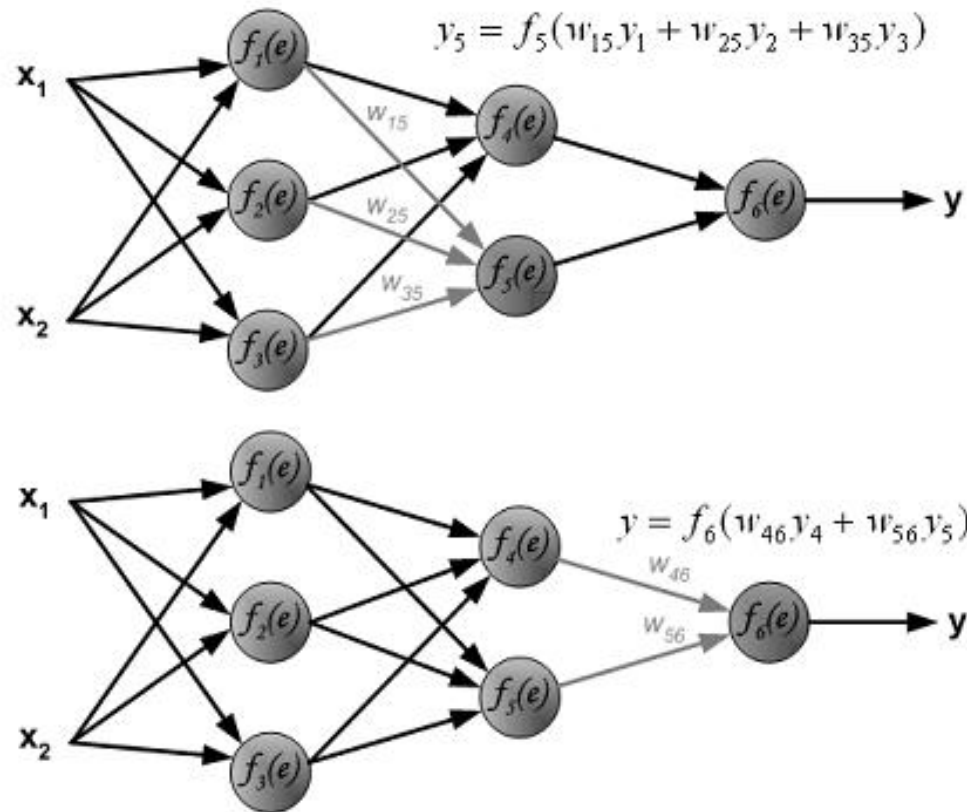
Backpropagation – Computação para frente

Exemplificação do passo 2:



Backpropagation – Computação para frente

Exemplificação do passo 2:



Backpropagation – propagação do erro

3 – Propagação do erro para trás

Como calcular o erro da rede?

Como propagar o erro para trás?

Backpropagation – propagação do erro

Como calcular o erro?

Depende em qual camada o neurônio se encontra:

✓ **Se pertencer à camada de saída**

$$\delta_j = f'(\text{net}_j) * (d_j - y_i)$$

Ao invés de usar a diferença absoluta, pode-se utilizar também o **erro médio quadrático (Mean Square Error – MSE) ou Root Mean Square Error – RMSE)**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (S_i - O_i)^2}$$

Backpropagation – propagação do erro

Vamos ver um exemplo o cálculo do MSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

X	Y	Saída desejada (XOR)	Saída real da rede	Erro
0	0	0	0,34	$(0 - 0,34)^2 = 0,1156$
0	1	1	0,57	$(1 - 0,57)^2 = 0,1849$
1	0	1	0,35	$(1 - 0,35)^2 = 0,4225$
1	1	0	0,46	$(0 - 0,46)^2 = 0,2126$
			Soma	0,9356
			MSE (Soma/4)	0,2339

Backpropagation - propagação do erro

Se pertencer à camada intermediária:

$$\delta_j = f'(\text{net}_j) \times \sum_l \delta_l w_{lj}$$

onde,

w_{lj} = Pesos das conexões

δ_l = Erro da Camada posterior

Backpropagation - propagação do erro

Resumindo

A forma para se calcular o erro depende da camada onde se encontra o neurônio

$$\delta_l = \begin{cases} f'_a e_l, & \text{se } n_l \in c_{sai} \\ f'_a \sum w_{lk} \delta_k, & \text{se } n_l \in c_{int} \end{cases}$$

Backpropagation - propagação do erro

Ou seja, como os valores dos erros são conhecidos apenas para os neurônios da camada de saída, o erro para os neurônios da camada intermediária precisam ser **estimados**

Backpropagation – Importância do gradiente

Importância do gradiente

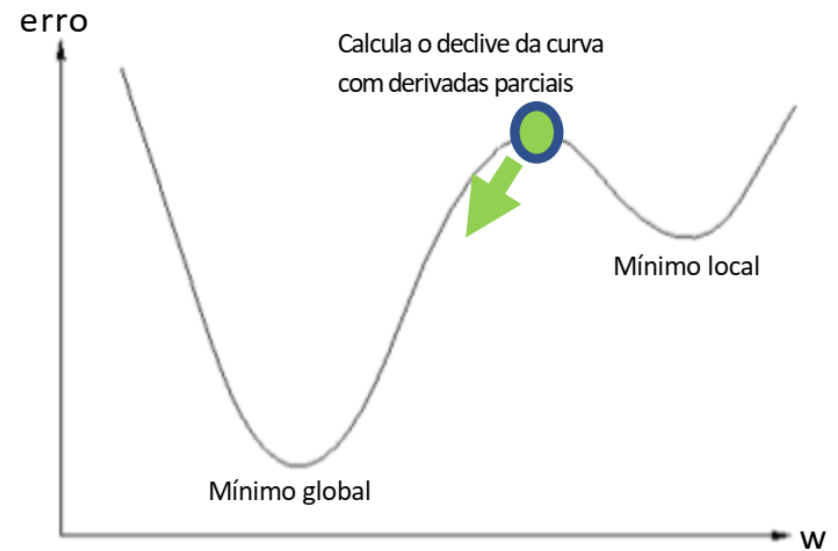
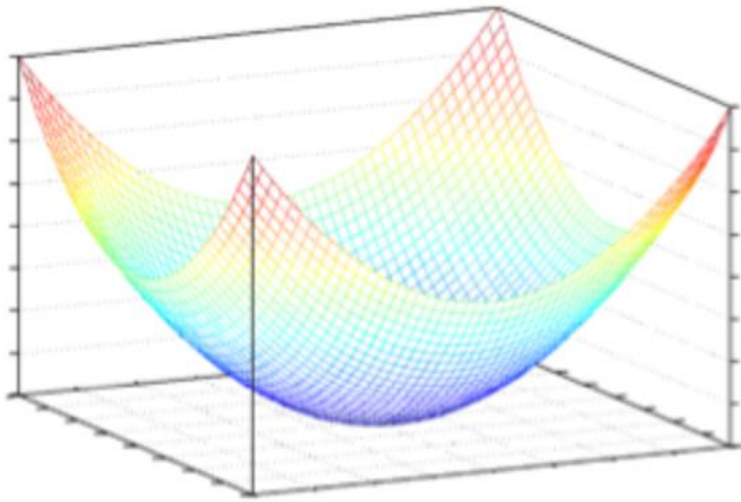
$$\min C(w_1, w_2, \dots, w_n)$$

O objetivo é calcular a derivada parcial para mover para a direção do gradiente

Ou seja, o objetivo é encontrar a combinação de pesos que **o erro é o menor possível**

Gradiente é calculado para saber quanto **ajustar os pesos**

Backpropagation – Importância do gradiente



Backpropagation – Importância da derivada

A derivada parcial define o ajuste dos pesos, utilizando o gradiente descendente da função de ativação.

Essa derivada mede a contribuição de cada peso no erro da rede para a classificação de um dado objeto x .

Se essa derivada para um dado peso for **positiva**, o peso está provocando um **aumento da diferença** entre a saída da rede e a saída desejada. Assim, sua magnitude deve ser **reduzida** para **baixar** o erro.

Se a derivada for **negativa**, o peso está contribuindo para que a saída produzida pela rede seja mais próxima da desejada. Dessa forma, seu valor deve ser **aumentado**.

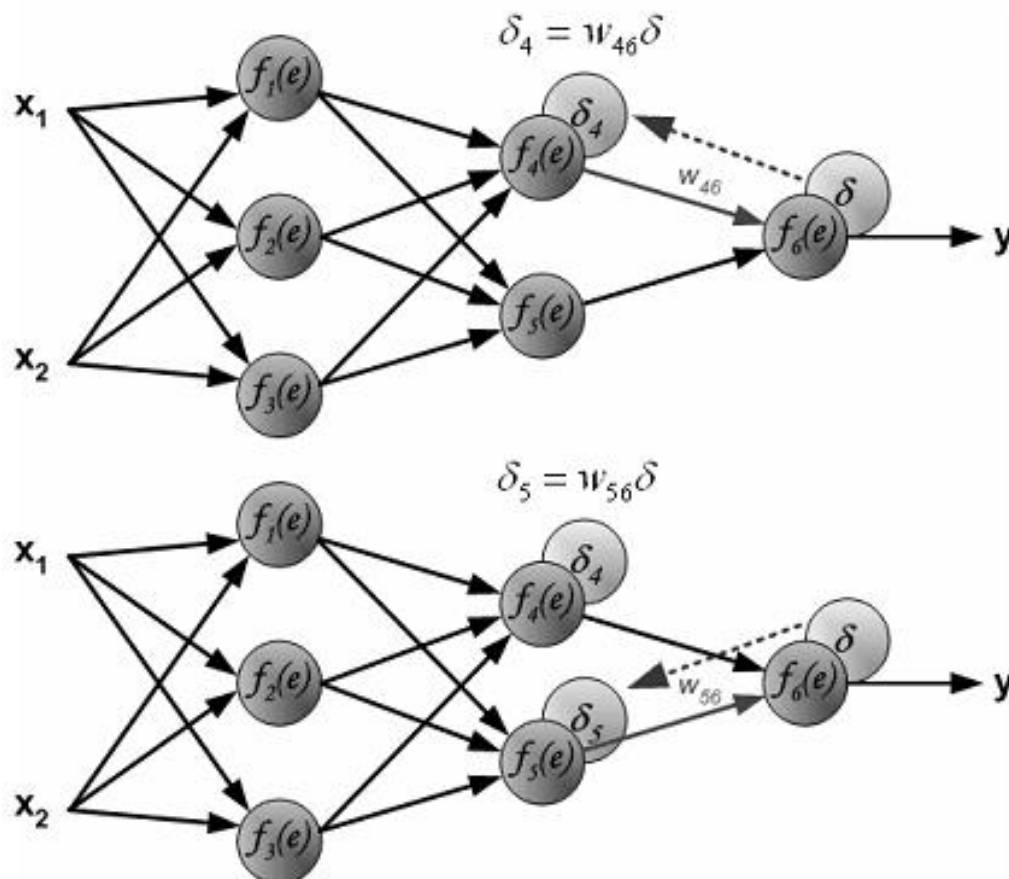
Se a Derivada é **0**, significa que estamos em um mínimo estável. Assim, nenhuma atualização nos pesos é necessária -> chegamos a um estado estável.

Backpropagation – Importância da derivada

Portanto, o sinal negativo da derivada é o que permite ao gradiente "subir" ou "descer" na direção certa — é isso que faz o gradiente descendente funcionar!!!!

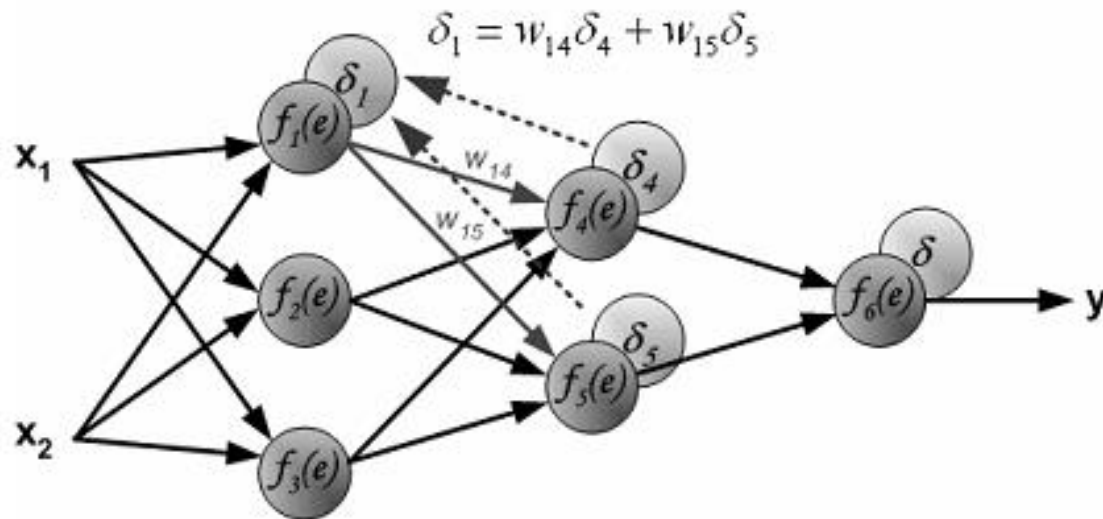
Backpropagation – propagação do erro

Exemplificação do passo 3 – propagação do erro:



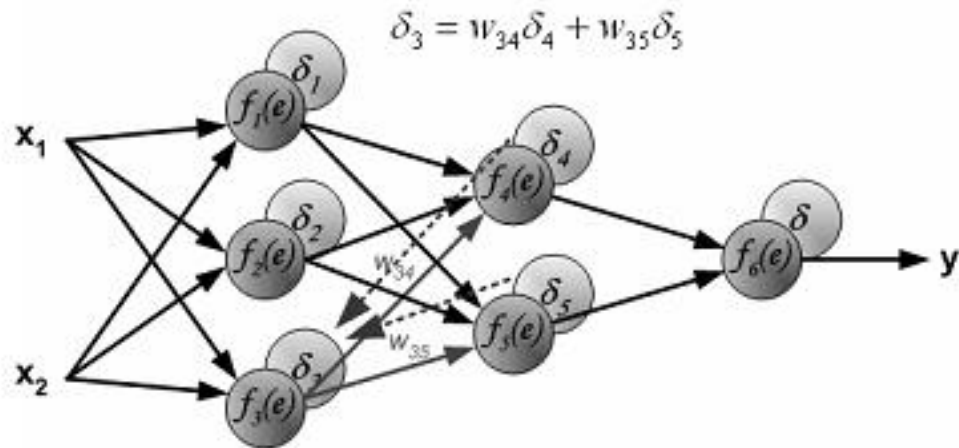
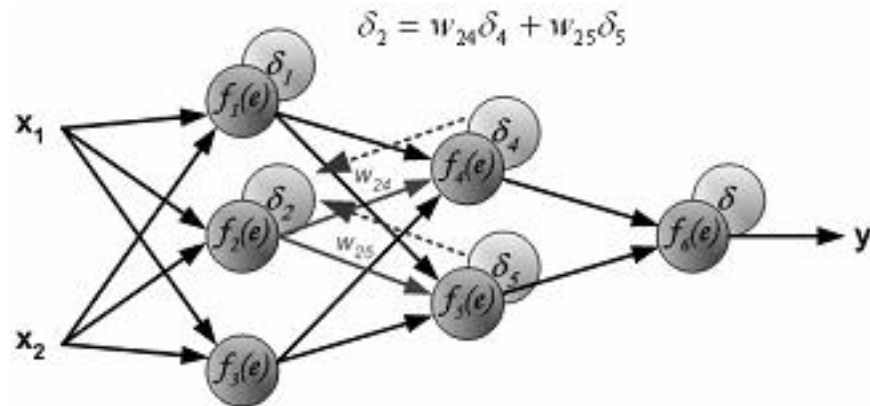
Backpropagation – propagação do erro

Exemplificação do passo 3 – propagação do erro:



Backpropagation – propagação do erro

Exemplificação do passo 3 – propagação do erro:



Backpropagation – Ajuste dos pesos

4 – Ajuste dos pesos

Após o cálculo dos erros é necessário corrigir os pesos das ligações entre os neurónios. O cálculo do novo peso é dado pela fórmula:

$$w_{ij}(t + 1) = w_{ij}(t) + Ta * x_i * erro$$

Onde,

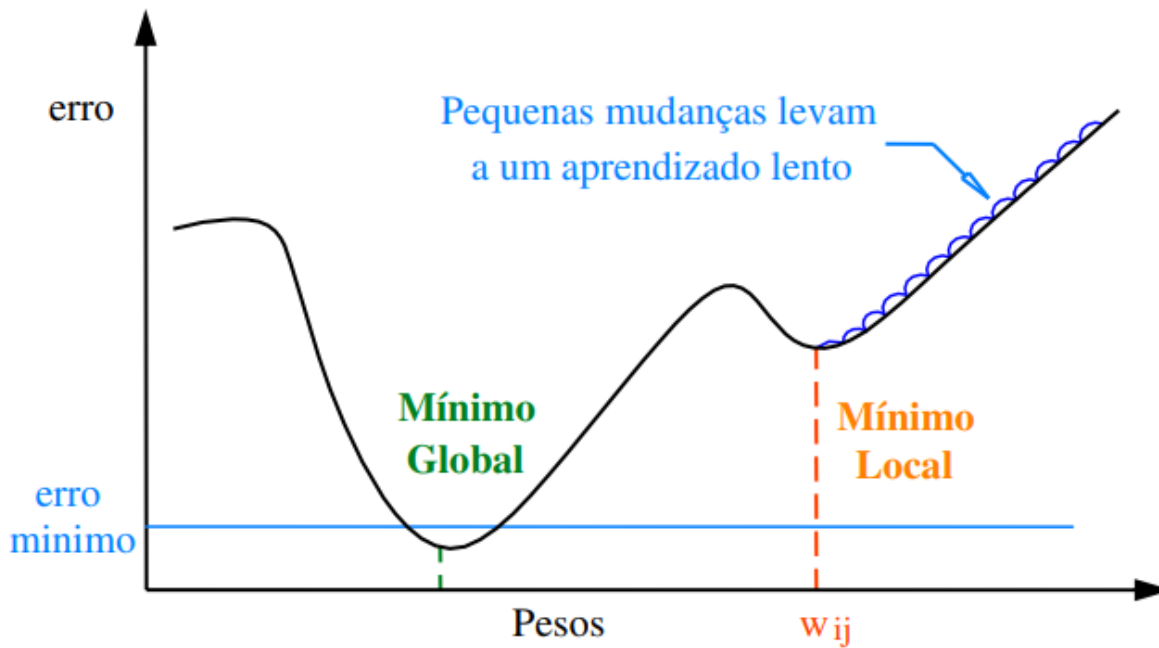
w_{ji} representa o peso entre um neurônio i e o j-ésimo atributo de entrada ou a saída do j-ésimo neurônio da camada anterior

$erro_j$ indica o erro associado ao i-ésimo neurônio

x_i indica a entrada recebida por esse neurônio (o i-ésimo atributo de entrada ou a saída do i-ésimo neurônio da camada anterior)

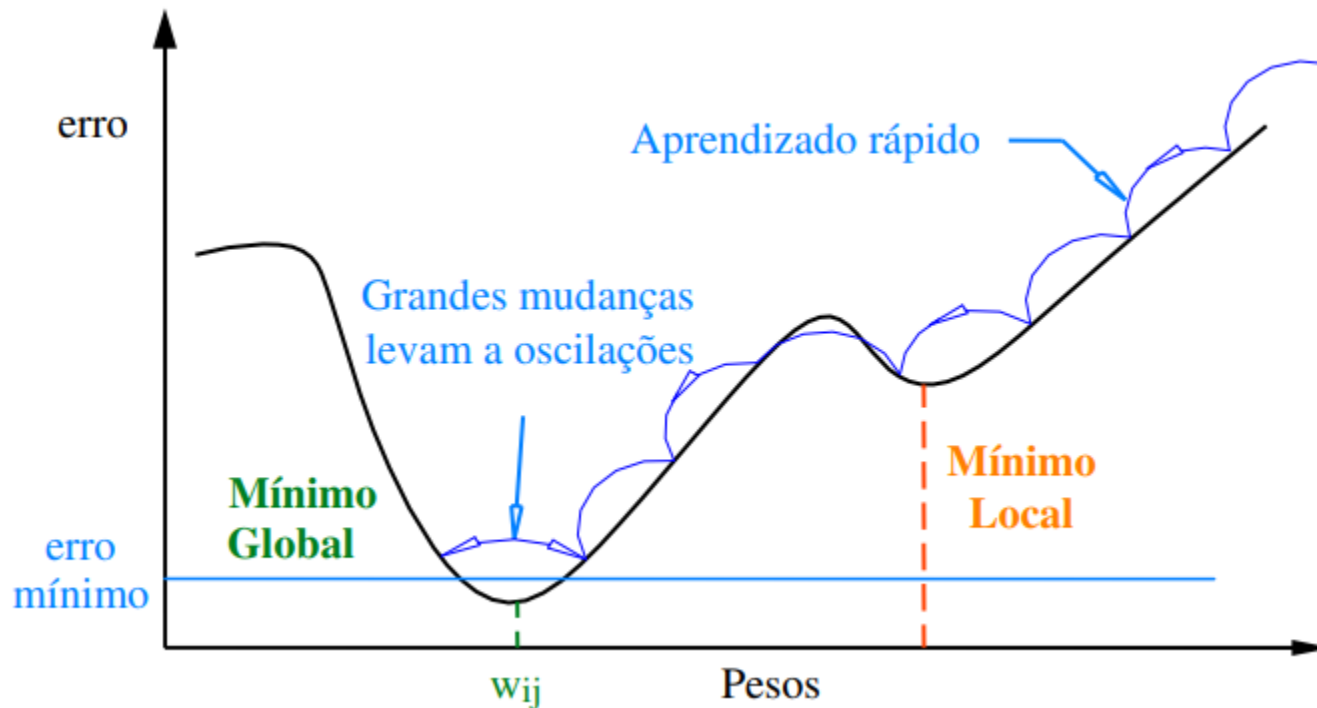
Backpropagation – Ajuste dos pesos e a taxa de aprendizado

Importância da taxa de aprendizado (sugestão que seja baixa)



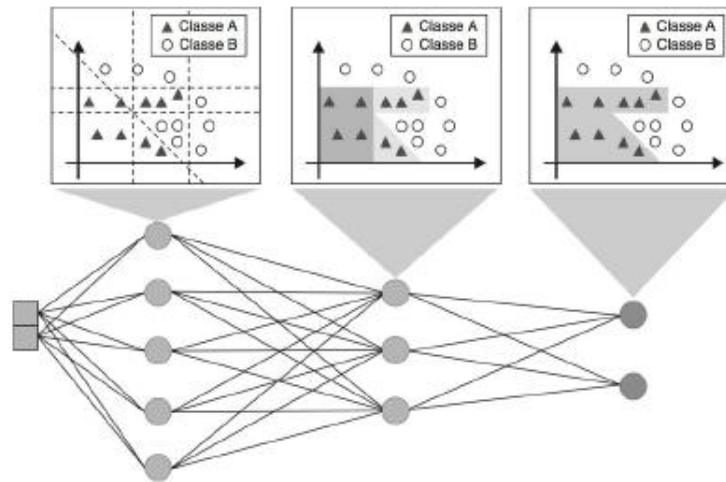
Backpropagation – Ajuste dos pesos e a taxa de aprendizado

Importância da taxa de aprendizado (valores altos de taxas de aprendizado)



MPL – papel dos neurônios das camadas ocultas

- ✓ Em uma MLP, cada neurônio realiza uma função específica.
- ✓ A função implementada por um neurônio de uma dada camada é uma combinação das funções realizadas pelos neurônios da camada anterior que estão conectados a ele.
- ✓ À medida que o processamento avança de uma camada intermediária para a camada seguinte, o processamento realizado se torna mais complexo.
- ✓ Na primeira camada, cada neurônio aprende uma função que define um hiperplano, o qual divide o espaço de entrada em duas partes.
- ✓ Cada neurônio da camada seguinte combina um grupo de hiperplanos definidos pelos neurônios da camada anterior, formando regiões convexas.
- ✓ Os neurônios da camada seguinte combinam um subconjunto das regiões convexas em regiões de formato arbitrário.
- ✓ É a combinação das funções desempenhadas por cada neurônio da rede que define a função associada à RNA como um todo.



Funções de ativação e suas derivadas

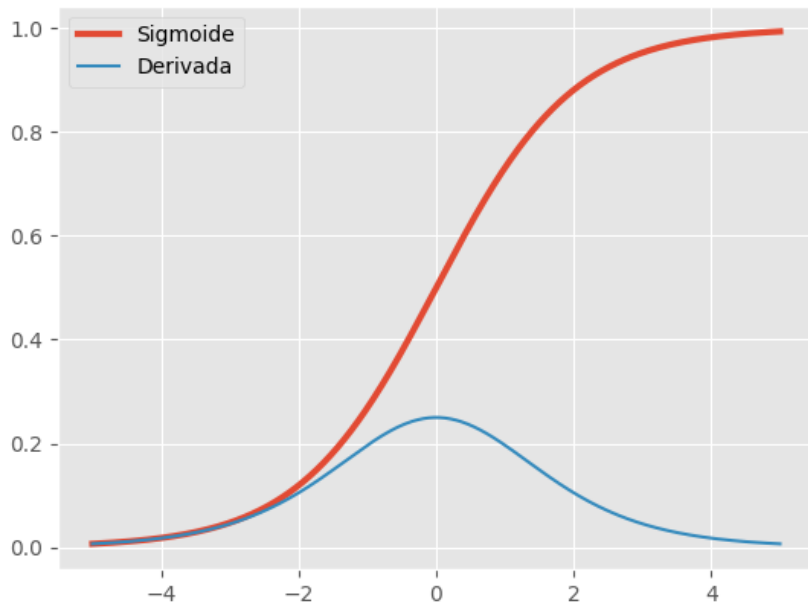
Para uma melhor compreensão das funções de ativação e suas derivadas, veja em:

<https://matheusfacure.github.io/2017/07/12/activ-func/>

Funções de ativação e suas derivadas

Sigmoide

$$\sigma(x) = \frac{1}{1 + e^x} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$



A derivada varia entre **0 e 0.25**, atingindo o máximo em $x = 0$.

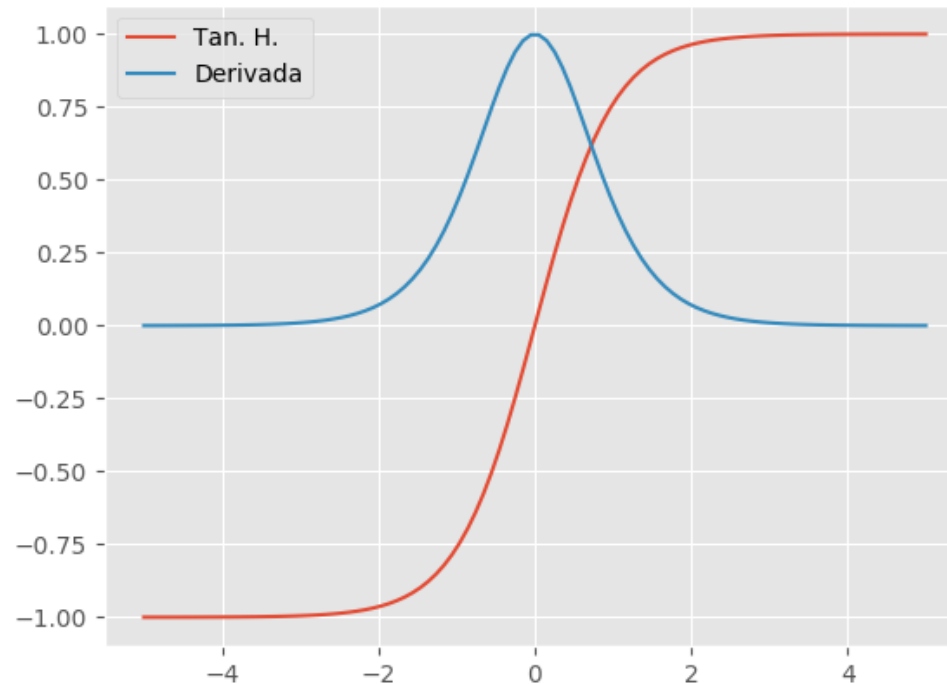
Observar o problema do **gradiente desaparecendo** (*vanishing gradient*)

Funções de ativação e suas derivadas

Tangente hiperbólica

$$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

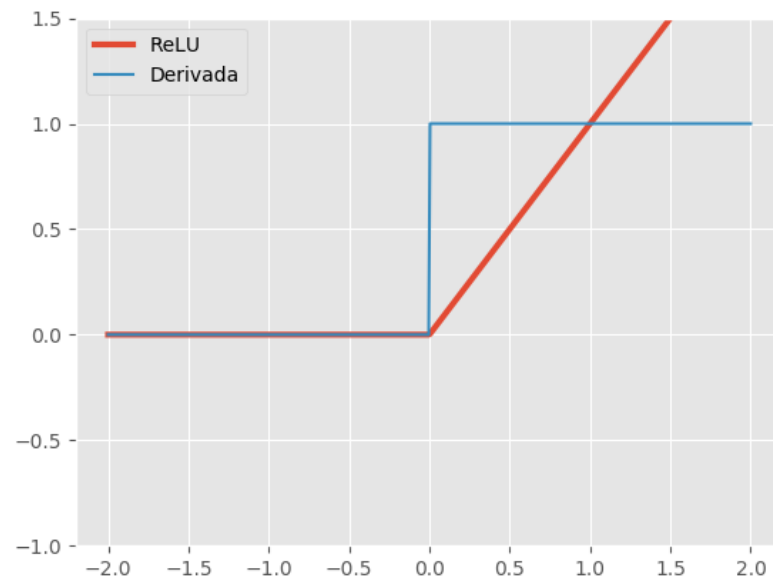
$$\sigma'(x) = 1 - \sigma(x)^2$$



Funções de ativação e suas derivadas

RELU

$$\text{ReLU}(x) = \max\{0, x\} \quad \text{ReLU}'(x) = \begin{cases} 1, & \text{se } x \geq 0 \\ 0, & \text{c.c.} \end{cases}$$



Funções de ativação e suas derivadas

Verifique outras funções:

SoftPlus, SoftMax, dentre outras

Todo algoritmo backpropagation utiliza a função de descendente gradiente?

Veja:

[Submitted on 25 Jan 2018]

A New Backpropagation Algorithm without Gradient Descent

Varun Ranganathan, S. Natarajan

The backpropagation algorithm, which had been originally introduced in the 1970s, is the workhorse of learning in neural networks. This backpropagation algorithm makes use of the famous machine learning algorithm known as Gradient Descent, which is a first-order iterative optimization algorithm for finding the minimum of a function. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient (or of the approximate gradient) of the function at the current point. In this paper, we develop an alternative to the backpropagation without the use of the Gradient Descent Algorithm, but instead we are going to devise a new algorithm to find the error in the weights and biases of an artificial neuron using Moore-Penrose Pseudo Inverse. The numerical studies and the experiments performed on various datasets are used to verify the working of this alternative algorithm.

<https://arxiv.org/abs/1802.00027>

Backpropagation - Considerações

- Para cada vetor de treinamento, o sinal deve ser propagado das entradas para as saídas para que o erro possa então propagar em sentido contrário e permitir o treinamento
- Há normalmente duas formas de atualização dos pesos:
 - 1- Atualiza-se os pesos a cada instância classificada incorretamente**
 - ✓ Stochastic gradient descent
 - 2 - Atualiza-se os pesos ao final de todas as instâncias apresentadas**
 - ✓ Batch gradient descent – e tem variação de ‘mini batch gradiente descent’, em que o usuário entra com o número de registros desejado
- Cada apresentação completa de todos os elementos do conjunto de treinamento e consequente ajuste de pesos é chamada *epoch*
- É aconselhável randomizar a sequência com que os vetores são apresentados à rede de uma *epoch* para a outra para acrescentar um componente estocástico ao treinamento e evitar ciclos limites indesejáveis na atualização dos pesos

Backpropagation – visualizações das curvas de aprendizado

- Veja este vídeo para **esclarecimento visual** sobre **Stochastic gradient descent** e **Batch gradient descent**

<https://www.youtube.com/watch?v=nrkGd4yNAEU>

- Veja outros vídeos sobre o processo de treinamento:

<https://www.youtube.com/watch?v=vH4eSfGkJhQ>

<https://www.youtube.com/watch?v=9g6CjmkiwRw>

Backpropagation e Geração dos pesos

Curiosidade: A geração de pesos iniciais seria sempre aleatório?

- Veja em: <https://link.springer.com/article/10.1007/s10462-021-10033-z>

Home > Artificial Intelligence Review > Article

A review on weight initialization strategies for neural networks

Published: 28 June 2021
Volume 55, pages 291–322, (2022) [Cite this article](#)

[Download PDF](#) 

 Access provided by Pontifícia Universidade Católica de PUC MG



Artificial Intelligence Review

[Aims and scope](#) →

[Submit manuscript](#) →

Outras formas de geração de pesos!

Heurísticas para número de neurônios

• **Regra do valor médio** - De acordo com esta fórmula o número de neurônios da camada escondida é igual ao valor médio do número de entradas e o número de saídas da rede, ou seja:

$$q = \frac{p + M}{2}$$

- **Regra da raiz quadrada** - De acordo com esta fórmula o número de neurônios da camada escondida é igual a raiz quadrada do produto do número de entradas pelo número de saídas da rede, ou seja:

$$q = \sqrt{p \cdot M}$$

Heurísticas para número de neurônios

- **Regra de Kolmogorov** - De acordo com esta fórmula o número de neurônios da camada escondida é igual a duas vezes o número de entradas da rede adicionado de 1, ou seja:

$$q = 2p + 1$$

Outras sugestões:

- O número de neurônios deve estar entre o número de entrada e saída
- 2/3 do tamanho da camada de entrada, somado ao tamanho da camada de saída
- Menor que duas vezes o tamanho da camada de entrada

Exercício

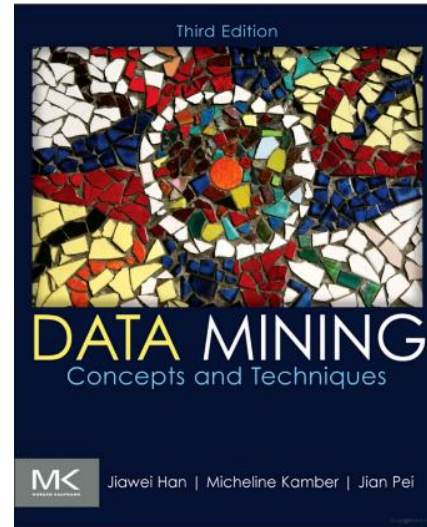
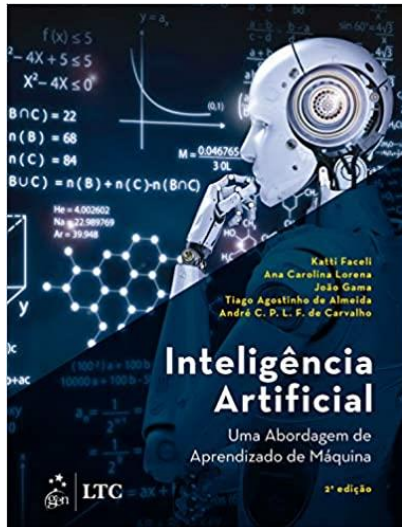
- Abra uma base de dados do seu interesse e investigue o use o Backpropagation
- Use o MLPClassifier from **sklearn.neural_network import MLPClassifier**

Exemplo: `rede_neural = MLPClassifier(max_iter=1000, verbose=True, tol=0.00001, solver = 'adam', activation = 'relu', hidden_layer_sizes = (3, 2, 1))`

- Investigue os hiperparâmetros:
hiddenLayers, batchSize, trainingTime, learningRate, momentum
- Investigue a relação entre learningRate e o trainingTime
- Investigue o desempenho da rede alterando o número de neurônios e camadas.

Referência

Veja capítulo 7 do livro texto adotado na disciplina



1. <https://link.springer.com/article/10.1007/s10462-021-10033-z>
2. https://www.google.com.br/books/edition/Data_Mining_Concepts_and_Techniques/pQws07tdpjoC?hl=en&gbpv=1&printsec=frontcover